Fix the Program, Not the Schedule

Bitwise-Reproducible LLM Serving with Every Optimization On

Haocheng Lu

November 2026

Abstract

Production LLM engines are not reproducible: the same request returns different tokens and log-probabilities depending on batch composition, prefix-cache state, chunking, and whether a token came from decode or prefill — and an RL trainer that rescores the rollout gets yet another answer. Today's deterministic modes buy reproducibility by switching optimizations off, and give up half their throughput or more doing so.

We show that determinism does not have to cost throughput. Instead of constraining the *schedule*, we fix the *program*: every query row and every output element executes one canonical floating-point sequence that depends only on positions and shapes, chosen so that its independent parts are exactly the partitions fast kernels want. Attention becomes an online softmax over 32-key tiles inside absolute 512-key fold groups; projections become one fixed MMA chain per element, cut into shape-determined segments. Prefill, chunked prefill, prefix-cache hits and decode are then bitwise identical by construction, while tiling, pipelining, stream-K and even the tensor-parallel degree stay free.

Implemented in a Rust serving engine with radix caching, chunked prefill, CUDA graphs and CPU/GPU overlap all left on, the deterministic mode is **1.2% faster on average** than the same engine without determinism across 40 workloads, **2.0–3.7× faster** than vLLM's batch-invariant mode — the only other engine that passes our numeric checks — and on par with the latest vLLM release. A PyTorch trainer on the same kernels reproduces all 55,593 rollout log-probabilities bit for bit, making every importance ratio exactly 1. Outputs are bitwise identical across an A800 and an RTX 4090 D. The principle carries over to linear attention: Gated DeltaNet decodes with the trainer's own chunked program inside vLLM, at stock throughput.

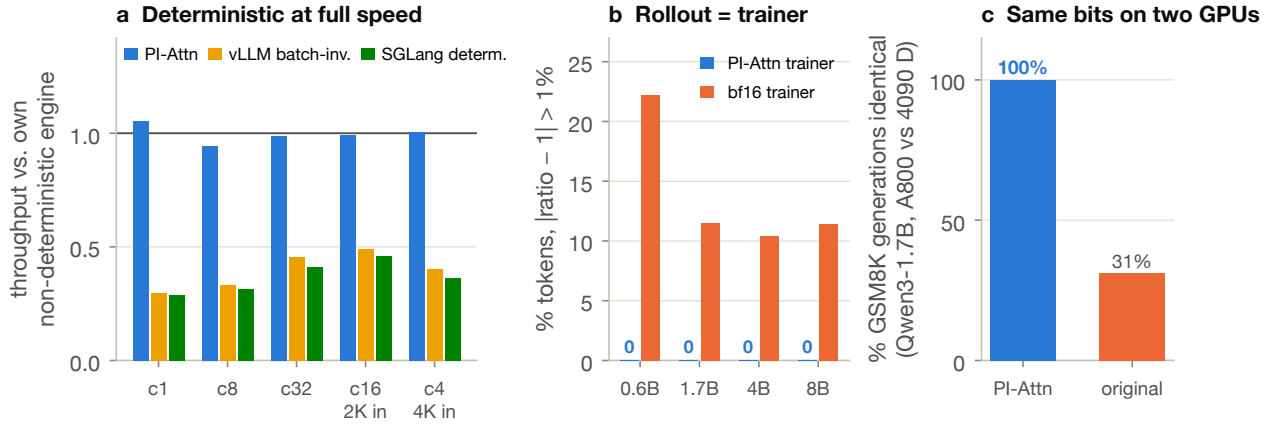


Figure 1. Determinism without the tax (A800). (a) Output throughput of three deterministic modes, each relative to its own engine without determinism, on Qwen3-1.7B: PI-Attn keeps full speed, while the batch-invariant modes of vLLM 0.17.1 and SGLang 0.5.5 keep 29–49%. (b) Rollout tokens whose trainer-side importance ratio deviates from 1 by more than 1%: none with PI-Attn's trainer forward, 10–22% with a standard bf16 trainer. (c) GSM8K generations bitwise identical across two GPU architectures.

1 Introduction

Ask a serving engine the same question twice at temperature 0 and you may get two different answers. Floating-point addition is not associative, and an engine re-partitions its reductions constantly: batch size changes GEMM tiling and split-K, load changes the number of KV splits in decode attention, chunked prefill moves where a prompt's attention is cut, a prefix-cache hit turns prefill into a cache read, and a token produced by the decode kernel is recomputed by the prefill kernel when the conversation continues. For evaluation this makes a benchmark score a sample rather than a value. For RL it means the rollout engine and the trainer disagree on every log-probability, and importance ratios drift from 1 on tokens the policy never changed.

The established fix is *batch-invariant* kernels [1]: pin one reduction order regardless of batch size by fixing or removing KV splits and pinning GEMM configurations. It works, at a steep price, because the optimizations it removes are exactly

the ones that adapt to load. SGLang reports a 34% slowdown on H100 [2]; on our A800 the batch-invariant modes of vLLM and SGLang keep 29–49% of their own throughput (Figure 1a). And prefill still disagrees with decode, because a prefill kernel rounds probabilities against a running maximum over the whole history while a decode kernel rounds against the maximum of its split.

Our insight: determinism is a property of a program, not of a schedule. Fix, for each query row and each output element, one canonical sequence of floating-point operations that depends only on positions and shapes. Any kernel that executes exactly that sequence produces the same bits, however rows are batched and whichever kernel runs them, and the sequence still leaves the schedule free wherever it has independent parts. The real question is therefore *which* sequence to fix — and that choice decides how fast deterministic kernels can be. We choose sequences whose free partitions are precisely the ones fast kernels already use:

- **Attention:** keys are cut at absolute positions into 512-key fold groups, each an online softmax over 32-key tiles, folded in a fixed order. A prefill kernel is a FlashAttention-2 kernel [3] that restarts its state every 512 keys; a decode kernel gives every group its own CTA; chunked prefill, cache hits and re-prefill execute the identical per-row sequence.
- **Projections:** each output element runs one MMA chain over K, cut into a number of segments fixed by the weight's shape. Tile shapes never enter the chain, so an autotuner can pick any tile, pipeline or stream-K split without changing a bit, and the segments are a deterministic split-K exactly where decode needs one.

The result is a serving engine in which every optimization stays on and determinism costs no throughput. We make five contributions:

1. **Canonical programs** (§2): serving determinism as a per-row program with free partitions at its associativity points, with invariance lemmas and an explicit numerical contract stating which parameters define the bits.
2. **PI-Attn** (§3): Ampere/Ada attention kernels for prefill, extend and decode that share one program, and a family of row-invariant GEMMs — streaming, CUTLASS, deterministic stream-K and segment-parallel kernels — that are bitwise identical to each other.
3. **A deterministic serving engine** (§4, §5) that is 1.2% faster on average than its non-deterministic self, 2.0–3.7× faster than the only other engine that passes our checks, and bitwise reproducible across load, cache state, configuration, restarts and GPU architectures.
4. **Rollout equals trainer** (§5.2): a PyTorch trainer forward on the same kernels reproduces every rollout log-probability bit for bit, whatever its micro-batch packing.
5. **Beyond softmax attention and one GPU** (§6): exact Gated DeltaNet decode inside vLLM at stock throughput with speculative decoding for free, and tensor-parallel degree invariance by making the declared maximum degree part of the program.

2 Canonical programs

Guarantee. For a fixed GPU architecture, engine build and model weights, every response is a bitwise function of the request's tokens, sampling parameters and seed. Batch composition, concurrency, cache state, chunking, decode versus prefill, CUDA graphs, page size and restarts do not enter; with a declared $t_{\max}$, neither does the tensor-parallel degree of the projections (§6.2). We rely on four hardware properties: (A1) an MMA instruction is a deterministic function of its operands and accumulator; (A2) `ex2.approx` and `lg2.approx` are deterministic; (A3) the compiler does not reassociate arithmetic written with round-to-nearest intrinsics; (A4) an MMA with an accumulator other than -0 does not return -0.

2.1 Attention

For query row i at position p, scores $s_j = (q_i \cdot k_j) \cdot \text{scale} \cdot \log_2 e$ are computed by MMA over the head dimension in ascending 16-wide steps and masked to $-\infty$ for $j > p$. Each fold group starts from the empty state $(m, d, o) = (-\infty, 0, 0)$ and runs, for its 16 tiles in ascending order,

$$m' = \max\left(m, \max_j s_j\right), \quad \sigma = 2^{m-m'}, \quad P_j = \text{bf16}\left(2^{s_j - m'}\right), \quad d \leftarrow \text{fma}\left(d, \sigma, \textstyle\sum_j P_j\right), \quad o \leftarrow \text{fma}(o, \sigma, +0) + PV,$$

with the row sum in one fixed order (and m' replaced by 0 while it is $-\infty$). Group partials fold in ascending order into (M, L, O): $M' = \max(M, m_g)$, $L \leftarrow \text{fma}(L, 2^{M-M'}, d_g 2^{m_g - M'})$, likewise for O, and the output is O/L (Figure 2). Two details make the program well defined on every path. A tile entirely past the query's position is an exact no-op (all $P = 0$, $\sigma = 1$, and the $+0$ rescale with A4 keeps o off -0), so prefill rows may sweep masked tiles that decode never

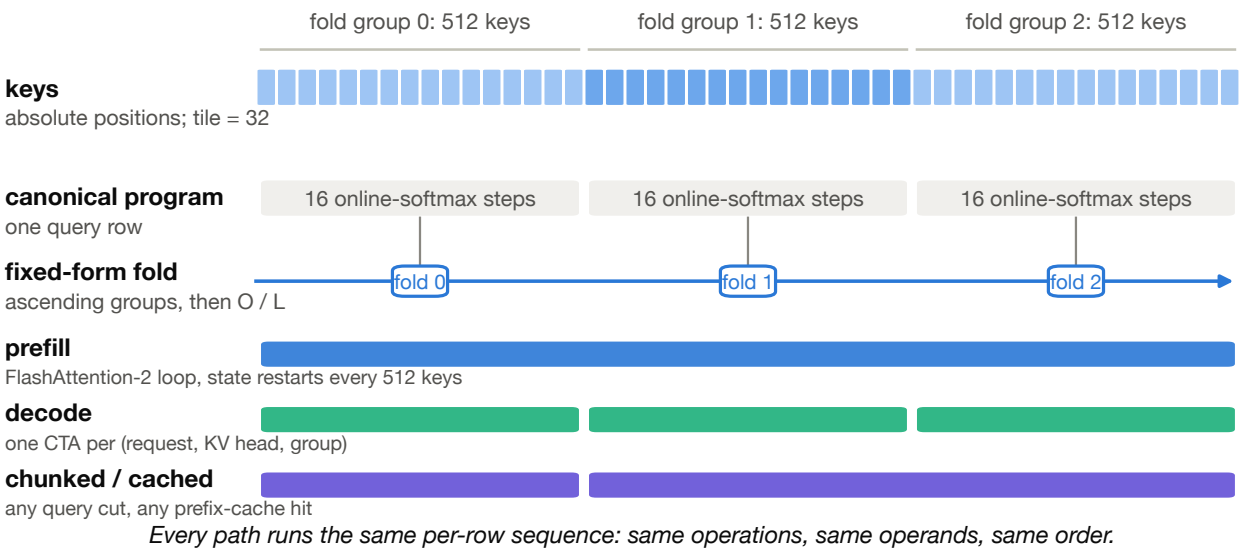


Figure 2. The canonical attention program. Keys are cut at absolute positions into 32-key tiles and 512-key fold groups. Each group runs 16 online-softmax steps from the empty state; group partials fold in ascending order. Prefill, per-group decode, and any chunked or cached split all execute this same per-row sequence.

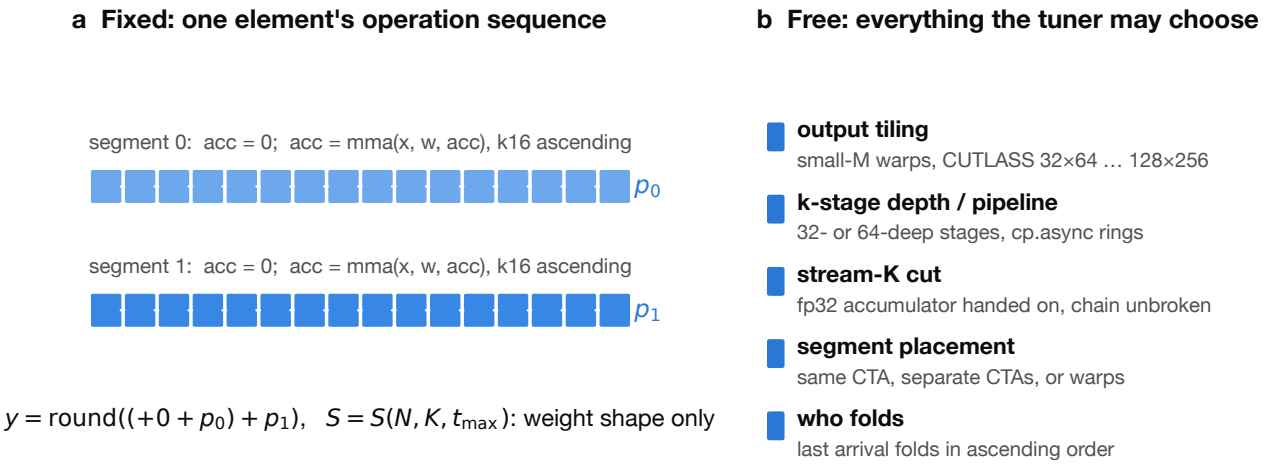


Figure 3. The canonical GEMM program. (a) Each output element runs one MMA chain per K segment, ascending from zero, and the segments fold from +0 in order; S depends only on the weight shape and the declared $t_{\max}$. (b) Everything else is free for the kernel and the autotuner.

sees. And folding into the identity is exact, so a kernel may keep the first group in registers or read it from memory.

Lemma 1. If a kernel executes this program for every row it owns, the output of row i is identical whatever rows share the launch, however groups map to CTAs, wherever chunked prefill cuts the query range, whether keys come from the prefix cache, and whether the row is decoded or prefilled. *Proof.* Every operation applied to row i, and every operand it reads, is fixed by p and the keys and values at positions $\leq p$. A kernel only chooses which rows, groups and tile slices it co-schedules, and splitting a tile across warps partitions S and O by columns without changing any element's chain. □

The 512-key group is the sweet spot: long enough that prefill runs an unmodified FlashAttention-2 inner loop, short enough that decode has 1–64 CTAs per KV head even at batch 1 for 0.5K–32K contexts. Among 512, 1024 and 2048 keys it was fastest for decode at batch ≤ 8, for shared prefixes and for short extends, at no cost to long prefill.

2.2 GEMM

K is cut into $S = S(N, K, t_{\max})$ equal segments; segment s accumulates its 16-wide k steps in ascending order by MMA from zero into p_s, and $y = \mathrm{round}\big((\cdots((+0 + p_0) + p_1)\cdots) + p_{S-1}\big)$ (Figure 3). With $t_{\max} = 1$, $S = 2$ for $N \le 1024$, $K \ge 2048$, $K \equiv 0 \pmod{512}$ — Qwen3-0.6B's o and down projections — and $S = 1$ otherwise.

Lemma 2. Any tiling of M and N, any assignment of (tile, segment) pairs to CTAs or warps, any choice of which CTA folds, and any cut of a segment's chain between workers that hands on the fp32 accumulator yield identical rows.

Segments are *deterministic split-K*. Split-K is nondeterministic in existing engines because the split count follows M and the SM count and partials combine in arrival order. Fixing S by the weight and the fold by segment index removes both, and spends segments only where decode needs them: narrow projections with long K.

2.3 The numerical contract

Table 1. The numerical contract. The left column is part of the model's numerical definition, like its dtype; nothing in the right column can change a bit.

fixed: defines the bits	free: chosen per launch
`mma.sync.m16n8k16`, bf16 in, fp32 accumulate	tile shape, warps per CTA, CTA count
ascending 16-wide k steps per segment, from zero	k-stage depth, pipeline and `cp.async` ring sizes
$S(N, K, t_{\max})$ and the left fold from +0	which CTA or warp runs or folds a segment
32-key tiles, 512-key groups, the fold form	stream-K cut points (fp32 accumulator handed on)
explicitly rounded softmax, fold and log-softmax	rows per CTA, group-to-CTA map, split count
declared tensor-parallel maximum $t_{\max}$	actual tensor-parallel degree t

$t_{\max}$ belongs to the contract. Segments exist because narrow weights need them, and a column shard at large t is as narrow as a small model's projection (§6.2), so S depends on the declared maximum degree. Two deployments that declare different $t_{\max}$ compute different bits, exactly as two dtypes would; one that fixes $t_{\max}$ gets identical bits at every degree. A launch never derives S from its local shard: every launch receives the whole weight's S. The engine fixes $t_{\max} = 1$ and exposes no knob to change it.

Every kernel runs the same instruction-level program. In the descriptor view of Yang et al. [4], a GEMM's bits are set by its accumulating instruction, the order and grouping of its k steps, its K remainder, and the precision and order of any merge. Our kernels fix each of these by construction. Hand-written kernels issue the single PTX instruction `mma.sync.aligned.m16n8k16.row.col.f32.bf16.bf16.f32`, and the CUTLASS tiles are instantiated with instruction shape `GemmShape<16, 8, 16>`, which lowers to the same instruction. Every kernel refuses a launch whose segment length is not a multiple of its k tile, so no remainder exists. Chains stay in fp32 registers, stream-K stores and reloads the fp32 accumulator bit-exactly, segment folds are explicit fp32 additions in ascending order, and nothing is built with fast-math. The test suite checks every kernel against two references on every output row of every projection shape at 14 values of M, and mutants prove that the check bites (§5.4).

2.4 Composition

RMSNorm, q/k-norm, RoPE, SiLU-and-mul and the KV store are row-wise or elementwise with fixed in-row orders. The sampled token's log-probability is a new row reduction, so we fix it explicitly: one 1024-thread CTA per row, strided per-thread passes for the maximum and for the sum of $2^{(z-m)\log_2 e}$, a fixed shared-memory tree, and only explicitly rounded operations, so a trainer that links the same header computes the same value. Sampling draws from per-row Philox streams keyed by (seed, position).

Theorem. In deterministic mode every activation, KV entry, logit and log-probability of token t is a function of tokens $0..t$ only, and every sampled token a function of tokens $0..t$ and the seed.

3 PI-Attn kernels

All kernels target sm_80/sm_89 and share one header holding the online step, the folds (built from `__fmaf_rn`, `__fmul_rn` and `ex2.approx`) and the finish. Kernels choose how work maps to warps and CTAs; none changes an element's sequence. Appendix B has the details.

Prefill and extend follow FlashInfer's scaffold [5]: 4 warps × 16 query rows per CTA covering one KV head's GQA

group, 128-byte-swizzled shared-memory tiles, and a software pipeline that computes $QK_{t+1}^{\top}$ while tile t runs its online step and PV. Each warp folds finished groups into a second register state. On sm_80 a variant keeps Q in registers and runs 8 warps per CTA for long and batched prefills (5–8% faster).

Decode is a single kernel with one 4-warp CTA per (request, KV head, group). Warps split each tile's scores and output columns; a request with one group finishes in place, otherwise the last warp to arrive folds every group in ascending order. The grid depends only on batch size and page-table width, so CUDA-graph replay needs no host planning.

The row-invariant GEMM family realises Lemma 2 in five kinds of kernel, all bitwise identical, among which an autotuner picks per (N, K, M): small-M streaming kernels with deep `cp.async` pipelines; balanced kernels that split (tile, segment) work evenly over the SMs; CUTLASS tiles from 32×64 to 128×256 with 32- or 64-deep stages; segment-parallel and segment-split kernels for segmented shapes; and **deterministic stream-K**. Stream-K divides the (tile, k-step) pairs of all tiles evenly over the SMs. When a tile's chain is cut between workers w and $w+1$, w stores its fp32 accumulator and $w+1$ continues *the same chain* from it, so the result is bit-identical to the uncut chain: split-K's load balance without split-K's reduction. Each worker runs its handed-off prefix first and waits only on its lower neighbour's prefix, which never waits, so with all CTAs resident it cannot deadlock. Stream-K brings 1.7B's down projection at $M = 8$ to 22.7 µs against cuBLAS's 21.2, and makes it faster than cuBLAS inside the batch-1 decode step (25.94 ⟶ 20.71 µs).

4 A deterministic serving engine

We implement the mode in llm_infer_rust, a Rust/libtorch serving engine with a radix prefix cache, chunked prefill, mixed prefill/decode batches, decode and segmented-prefill CUDA graphs, and CPU/GPU overlap. `--deterministic` disables none of them. It routes attention to PI-Attn and every projection, including the LM head, to the row-invariant GEMM, and keys each sampled row's Philox stream by (request seed, position); requests without a seed get a random one, echoed back so any output can be replayed. With `logprobs: 0` a request also receives each sampled token's log-probability from the fixed-order kernel, without disturbing the overlap pipeline. At startup, after graph capture, the engine tunes every projection shape for every CUDA-graph batch bucket on random operands, with every launch streaming weights from DRAM as in a real forward (4.0–15.5 s for 0.6B–8B on the A800).

5 Evaluation

Setup. One A800-SXM4-80GB (sm_80; CUDA 12.6, PyTorch 2.13, FlashInfer 0.6.18) at stock clocks, Qwen3 0.6B–8B in bf16. *Original* is llm_infer_rust with FlashInfer attention and cuBLAS; *PI* is the same binary with `--deterministic`; both run with radix caching, chunked prefill, mixed batches, CUDA graphs and overlap. Baselines on the same GPU: SGLang 0.5.5.post3 with `--enable-deterministic-inference`, vLLM 0.11.2, and vLLM 0.17.1 with `VLLM_BATCH_INVARIANT=1`. An RTX 4090 D (sm_89) provides the cross-GPU comparison.

5.1 Bitwise reproducibility

We built *detcheck*, a black-box checker for any OpenAI-compatible server that compares each response's token ids and the fp32 bits of every token's log-probability across concurrency, shared-prefix batches, decode versus re-prefill, prefix-cache hits, seeded sampling and 200 repeats.

PI-Attn passes every check (Figure 4), on all four models. vLLM 0.17.1's batch-invariant mode also passes the numeric checks; its one failure, seeded re-prefill, comes from its sampler keying random streams by request start rather than by position. SGLang's deterministic mode still changes log-probabilities with concurrency and across shared prefixes, and no engine besides PI and vLLM's batch-invariant mode makes decode agree with prefill. Log-probabilities are the stricter test: at token level the original engine looks consistent on decode versus re-prefill and on cache hits, but with log-probabilities all 24 re-prefilled requests differ. Across server configurations — chunk size 512, chunking off, CUDA graphs off, 4 running requests, overlap off, page size 32, and a plain restart — PI is identical every time on Qwen3-0.6B, while the original engine fails 11–13 of 32 comparisons in each, including the restart.

Evaluation becomes a function of the model. On GSM8K at client concurrency 1 versus 16, 888 of 1319 Qwen3-1.7B generations change with the original engine and its score moves by 0.68 points, more than the gap between the two modes; with PI, none change, on 1.7B and 8B alike. Accuracy is unchanged: top-1 agreement with an fp32 reference is 98.91–99.61% for PI and 99.14–99.53% for the original engine, differing by at most 5 positions per model, all at near-ties (Appendix A).

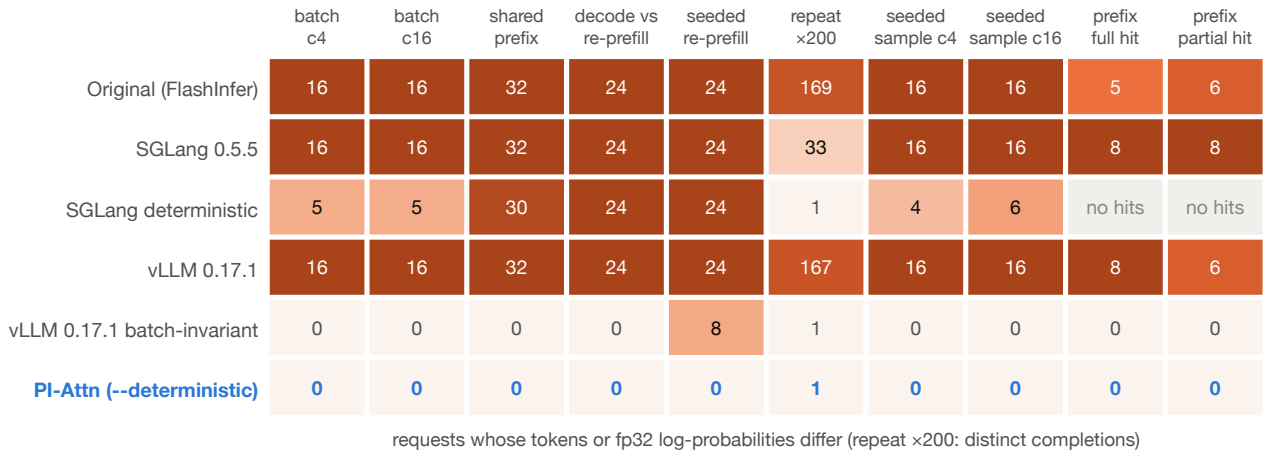


Figure 4. detcheck on Qwen3-1.7B (A800): requests whose tokens or log-probabilities differ from the reference. *Decode vs re-prefill* resubmits the first k generated tokens as prompt and compares the continuation; *repeat* counts distinct completions of one prompt over 200 runs. SGLang's deterministic mode disables the radix cache on this path, so it has no cache hits to compare.

Same bits on two GPU architectures. Every PI kernel output in an 80-case GEMM and 5-case attention suite, and every PI GSM8K generation, is bitwise identical between the A800 (sm_80) and the RTX 4090 D (sm_89). cuBLAS agrees on 14 of 80 cases, FlashInfer on 1 of 5, and the original engine's generations differ on 906–910 of 1319 (Figure 1c).

5.2 Rollout log-probabilities equal the trainer's

The trainer side is the engine's Qwen3 forward in PyTorch on the engine's own kernels: PI GEMM, PI-Attn causal prefill over a paged cache, RMSNorm, RoPE and the same log-softmax. We sample 64 GSM8K rollouts per model from the engine at concurrency 16 (temperature 1, per-request seeds) and rescore them with this trainer, each sequence alone and in two random packings of 2–8 sequences per micro-batch, and with HF transformers in bf16.

Every one of the 55,593 rollout tokens gets exactly the engine's log-probability, whichever packing the trainer uses, so every importance ratio is exactly 1 (Figure 1b). The engine produced those tokens by decode, in batches of up to 16 under chunked prefill; the trainer recomputes them in one prefill per micro-batch — Lemma 1 at work. The standard bf16 trainer matches on at most 41% of tokens and misses the 1% ratio band on 10–22%, with ratios from 0.49 to 2.51. Truncated and masked importance sampling exist to absorb exactly this noise; PI removes it at the source.

5.3 Performance

Determinism costs no throughput end to end (Figure 5a). Across 40 workload points — four models, five workloads, greedy and sampled — PI runs at −5.9% to +14.2% of the original engine, **+1.2% on average** (geometric mean). At batch 1 it is faster on 1.7B, 4B and 8B by 5.7–13.8%: PI's single-kernel decode replaces FlashInfer's decode-plus-merge, and stream-K beats cuBLAS on 1.7B's down projection. Batched workloads sit within ±2.5% except 1.7B at 8 requests (−5.8%) and three 0.6B points (+4.3% to +7.1%). Ablating each half shows that the attention alone gains 1.8–4.7% at batch 1 and never costs more than 0.3%, while the GEMM costs at most 1.6% at 32–128 requests (Appendix A).

Against other engines (Figure 1a, Appendix A), PI matches the latest vLLM release without determinism — within −2.8% to +10.4% of vLLM 0.17.1 — and beats SGLang on all five workloads. The deterministic alternatives pay heavily: vLLM 0.17.1's batch-invariant mode runs at 30–49% of vLLM's own throughput, so **PI is 2.0–3.7× faster** than the only other engine that passes our numeric checks; SGLang's deterministic mode, which also disables the radix cache, runs at 29–46% of SGLang's. (vLLM documents its batch-invariant mode for compute capability ≥ 9.0, so its cost on Hopper may differ.)

Where PI pays: long prefills on full-rate tensor cores (Figure 5b). Time to first token rises 12–18% on 16K–30K-token prompts and up to 6.3% on 2K–4K prompts, and prefill-heavy 0.6B at 64 requests loses 8.0% of throughput. The A800 runs fp32-accumulate MMA at full rate, which exposes the two costs the canonical program imposes on prefill: the serial per-32-key online step between the $QK^\top$ and PV MMAs, and the fragment traffic of 16-row warps. On the RTX 4090 D, whose half-rate MMA hides both, the same design keeps time to first token within +1.6% on every long

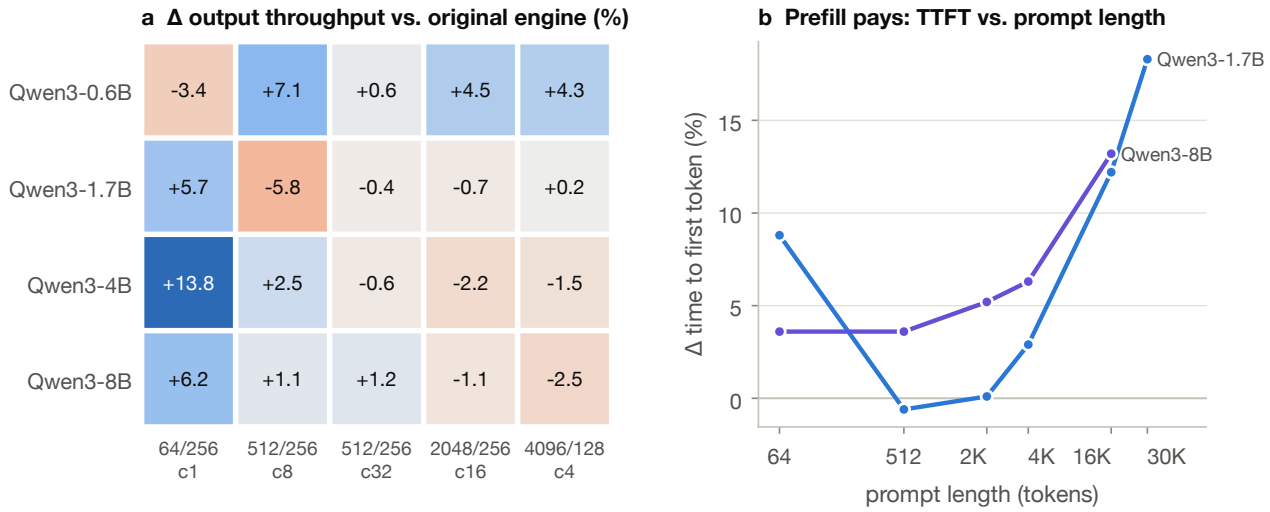


Figure 5. (a) End-to-end output throughput of PI relative to the original engine (greedy; median of three interleaved launches). (b) Change in time to first token with prompt length for the long-context workloads (each at its own concurrency).

prompt and is faster than the original engine on 18 of 20 points (Appendix C). This is the price of one program for every GPU: retuning the tile or group size per GPU would change the bits.

Kernels. Timed kernel against kernel by replaying CUDA graphs over a KV ring larger than L2, PI's decode takes 0.91–1.06× the time of the faster FlashInfer decode at batch 8–32, 0.73–0.85× with shared prefixes, and 1.03–1.10× at batch 1 with 8K–32K keys; against FlashInfer with its split size fixed at 2048, its batch-invariant configuration, PI is 5.2× faster at 32K keys. Prefill takes 1.12–1.80× FlashInfer's time. The best PI GEMM takes 0.78–1.37× cuBLAS's time at $M = 1$ and 0.43–1.33× at larger M; the gaps are on o and down projections, where cuBLAS uses split-K, which the canonical k order rules out (Appendix A).

5.4 The tests can fail

A bitwise check that never fails proves little, so we mutate kernels and require the tests to catch every mutant: a merge folding groups in descending order, prefill closing groups every 256 keys instead of 512, and a decode merge rounding twice instead of once — a one-ulp fp32 change, which forced us to strengthen the decode-versus-prefill check to cover every position (it now fails 60 times). cuBLAS fails the M-invariance check on 26 of 27 projections, and the fold check rejects a wrong segment count. Accuracy is asserted rather than printed: attention error against float64 must stay within 1.05× (mean) and 1.25× (maximum) of FlashInfer's, and PI's mean error is below FlashInfer's on every geometry; every GEMM output lies within the fp32 summation bound of the float64 product.

6 Beyond softmax attention and one GPU

6.1 Exact Gated DeltaNet decode

Hybrid models (Qwen3.5, Qwen3-Next, Kimi Linear) interleave attention with Gated DeltaNet (GDN) layers, which are *trained* with a chunked forward (fla's `chunk_gated_delta_rule`) and *served* with a recurrent kernel — two programs, so rollout and trainer disagree, and speculative decoding adds a third. The principle applies unchanged, with the trainer's chunk program as the canonical one. fla's chunk program is row-causal and resumable bitwise from any 64-token boundary state, so our decode kernel computes the current token's row of the chunk program from the last boundary state and the chunk's cached rows, folding the state once a chunk's 64th row is accepted. On Qwen3.5 with row-invariant dense ops, decode then equals the teacher-forced forward at all 724 scored positions, where recurrent decode flips the argmax at 0.8–1.2% of them.

Speculation is free. A verification window computes more rows of the same chunk program, and a chunk is folded only after its last row is accepted, so rollback needs no state copies, where vLLM keeps 1 + drafts fp32 states per request.

Inside vLLM 0.17.1, with exact GDN and PI-Attn's GEMM as vLLM's batch-invariant dense ops, decode equals prefill

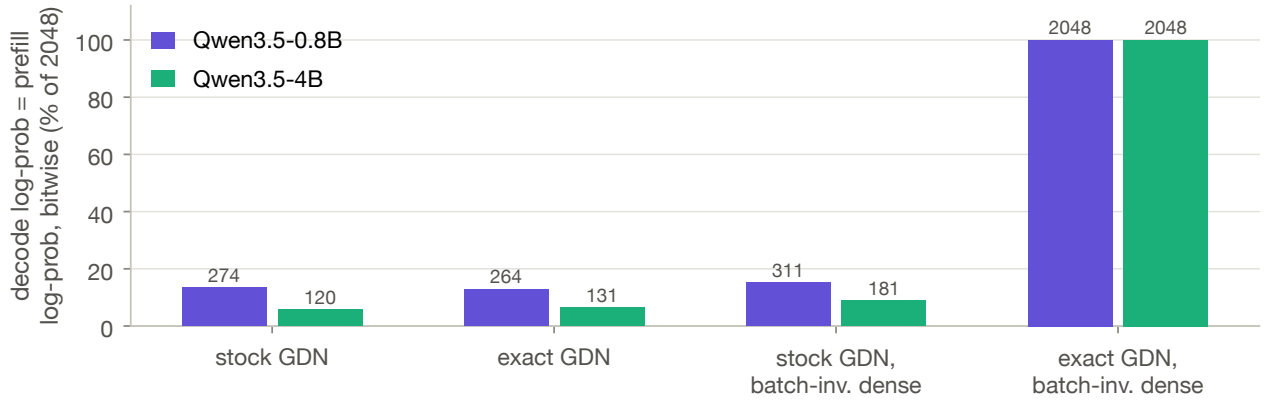


Figure 6. Decode log-probability equals prefill log-probability, bitwise, in vLLM 0.17.1 (2048 generated tokens per model). Only exact GDN together with batch-invariant dense projections makes decode the trainer's program.

on all 2048 generated tokens on Qwen3.5-0.8B and 4B (Figure 6), through chunked prefill, slot reuse, preemption and n-gram or MTP speculation, at **1.01× stock vLLM throughput** (1.08× with MTP, 0.96–1.03× elsewhere). The one exception is MTP combined with chunked prefill, where vLLM's own prompt log-probabilities change; the decode output is still the unspeculated run's. With vLLM's own batch-invariant matmul instead of PI's GEMM, the same configuration drops to 0.27× at batch 1: in vLLM, the cost of batch invariance lives in the dense projections, and PI's GEMM removes it.

6.2 Tensor-parallel degree as a canonical parameter

Megatron-style tensor parallelism splits the K sum of row-parallel projections across ranks and all-reduces the partials, so the bits depend on the degree t. We keep every projection column-parallel: each rank owns whole-K output columns and all-gathers copy bits, so each output element runs its $t = 1$ chain at any degree, including degrees that do not divide the head count. Narrow shards are bounded by chain latency rather than bandwidth, which is why S grows with the declared $t_{\max}$ (§2.3). With ranks emulated on one GPU, Qwen3-8B reproduces its $t = 1$ logits bitwise at $t = 2, 3, 4, 8$, while Megatron row-parallel o/down changes 77–87% of them; per-rank GEMM time is 1.02–1.26× the row shard's at $t = 2$ for 70B shapes. Measuring the extra all-gather traffic (1.63× the bytes for Llama-3-70B at $t = 8$) on a multi-GPU system is the next step.

7 Related work

Batch-invariant kernels [1] underlie SGLang's deterministic inference [2] and vLLM's `VLLM_BATCH_INVARIANT`, which pin splits and GEMM configurations. DeepSeek-V4 [6] drops split-KV and most split-K and engineers a second attention kernel to match the single-SM order, and a vLLM pull request [7] routes every row through the unsplit decode kernel at 0.84–0.91× throughput. These pin the partition or engineer one kernel to reproduce another; we leave the partition free and make prefill and decode agree by construction. **Deterministic attention** with fixed absolute splits appears in mlx-batch-invariant [8] (prefill 3.9–4.4× slower than stock) and a llama.cpp draft [9]. Vosti [10] verifies an ordered single-loop attention — our program with one unbounded group, which cannot split long decodes — and FoldAttention [11] declares the softmax reference ahead of the scan. Our contribution is the in-group program that FlashAttention-2 and group-parallel decode both execute as is, inside an engine with a prefix cache; FlashDecoding [12] and FlashInfer [5] merge split partials, but along schedule-dependent boundaries. **Bitwise GEMM.** Yang et al. [4] describe a GEMM's bitwise behaviour by a descriptor of its reduction order, show that output tiling leaves accumulation chains intact, reconstruct cuBLAS's descriptor, enforce orders in Triton and verify equivalence statically on PTX and AMDGCN. Our program is such a descriptor designed for serving: invariant to M and to tensor-parallel sharding, realised by a whole kernel family, and extended to attention, which [4] treats only theoretically. Cooper et al. [13] accumulate in explicit IEEE fp32 for cross-architecture identity; TBIK [14] orders tensor-parallel all-reduces; CoRun [15] pads to fixed shapes; LLM-42 [16] verifies and rolls back; reproducible summation [17] makes sums order-free at higher cost; and a recent stream-K for convolution gradients [18] folds partial tiles in a fixed order, where ours hands on the accumulator itself.

8 Scope and conclusion

The guarantee is per GPU architecture and build; across sm_80 and sm_89 the bits also agree in every case we compared, and guaranteeing identity on Hopper's `wgmma` would call for explicit IEEE accumulation as in [13]. The engine runs on one GPU with head dimension 128; FP8, MoE kernels and a full RL training loop are natural next steps.

Bitwise-reproducible serving does not require turning optimizations off. It requires choosing the floating-point program first and letting every kernel, partition and engine feature execute it. With attention cut into absolute 32-key tiles and 512-key groups and projections into shape-determined segments, batching, chunking, caching, decode, packing and even the tensor-parallel degree become schedule choices that cannot change a bit, and a trainer running the same program sees exactly what the rollout saw. Determinism, it turns out, need not cost throughput.

References

[1] H. He and Thinking Machines Lab, "Defeating nondeterminism in LLM inference." Thinking Machines Lab: Connectionism, 2025.

[2] The SGLang Team, "Towards deterministic inference in SGLang and reproducible RL training." LMSYS Blog, 2025.

[3] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv:2307.08691*, 2023.

[4] Z. Yang, N. J. Riasanovsky, W. Deng, and V. Sarkar, "Taming bitwise behavior in GPU kernels with tensor core: Black-box reconstruction, compiler enforcement, and static verification," *arXiv:2609.11356*, 2026.

[5] Z. Ye *et al.*, "FlashInfer: Efficient and customizable attention engine for LLM inference serving," *arXiv:2501.01005*, 2025.

[6] DeepSeek-AI, "DeepSeek-V4: Towards highly efficient million-token context intelligence," *arXiv:2606.19348*, 2026.

[7] vLLM contributors, "Bitwise batch invariance for DeepSeek-V4.1-Flash with CUDA graphs." vLLM pull request 62365, 2026.

[8] waseemh, "Mlx-batch-invariant: Batch-invariant kernels for MLX on Apple silicon." github.com/waseemh/mlx-batch-invariant, 2025.

[9] llama.cpp contributors, "Deterministic inference mode (CUDA)." llama.cpp pull request 16016 (draft), 2025.

[10] J. Qin, A. Du, D. Zhang, M. Lentz, and D. Zhuo, "Vosti: Specifying, implementing, and verifying deterministic LLM inference," *arXiv:2609.38981*, 2026.

[11] "FoldAttention: Declared-reference softmax for fast decode and deterministic backward," *arXiv:2609.33410*, 2026.

[12] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-decoding for long-context inference." Blog post, 2023.

[13] L. Cooper, S. Jeong, H. Jeon, J. Young, and H. Kim, "Accelerating the mitigation of LLM inference nondeterminism across GPU architectures," *arXiv:2609.25624*, 2026.

[14] Z. Zhang *et al.*, "Deterministic inference across tensor parallel sizes that eliminates training-inference mismatch," *arXiv:2511.17826*, 2025.

[15] "CoRun: Padding is simple and efficient for deterministic LLM inference," *arXiv:2608.14376*, 2026.

[16] "LLM-42: Enabling determinism in LLM inference with verified speculation," *arXiv:2601.17768*, 2026.

[17] J. Demmel and H. D. Nguyen, "Fast reproducible floating-point summation," in *IEEE symposium on computer arithmetic (ARITH)*, 2013.

[18] ROCm contributors, "Stream-K for implicit-GEMM convolution weight gradients." rocm-libraries pull request 13151, 2026.

Appendix A: Additional results

Table 2. Same-GPU baselines: greedy output tokens/s on Qwen3-1.7B (A800). The original and PI rows are a separate launch from Figure 5a (differences up to 1.8%). vLLM 0.11.2's batch-invariant mode is not batch-invariant on sm_80.

engine	64/256 c1	512/256 c8	512/256 c32	2048/256 c16	4096/128 c4
original	283	1894	5014	1878	539
PI	**298**	**1790**	**4951**	**1863**	**542**
SGLang 0.5.5	253	1768	4865	1837	530
SGLang deterministic	73	554	1993	843	192
vLLM 0.11.2	237	1599	4149	1694	508
vLLM 0.11.2 batch-invariant	244	1539	4083	1671	495
vLLM 0.17.1	270	1842	4890	1885	553
vLLM 0.17.1 batch-invariant	80	609	2228	921	223

Table 3. Component ablation: output throughput relative to the original engine (workload = input / output tokens / concurrency). The two halves roughly add.

model	workload	original tok/s	PI attention only	row-invariant GEMM only	both
0.6B	64 / 256 / c1	513.5	+3.3%	−6.1%	−2.9%
0.6B	512 / 256 / c32	7454.7	+1.3%	−0.8%	+0.4%
0.6B	512 / 256 / c128	11641.0	−0.3%	−1.4%	−1.7%
1.7B	64 / 256 / c1	282.7	+1.8%	+3.9%	+5.9%
1.7B	512 / 256 / c32	5080.5	+1.1%	−1.6%	−0.6%
8B	64 / 256 / c1	89.9	+4.7%	+1.4%	+6.8%
8B	512 / 256 / c32	1813.7	+0.5%	+0.5%	+1.1%

Table 4. Long-context, shared-prefix and high-concurrency workloads, PI vs. original (A800; median of three interleaved launches). Throughput is output tokens/s, or requests/s for the 16- and 4-token outputs. The 8B 16K row produces one token per request because 16384 + 256 exceeds the server's context limit.

model	workload (in / out / concurrency)	Δ throughput	Δ TTFT
1.7B	64 / 256 / c1	+5.6%	+8.8%
1.7B	512 / 256 / c32	−0.3%	−0.6%
1.7B	2048 / 16 / c8	−1.7%	+0.1%
1.7B	4096 / 4 / c4	−2.7%	+2.9%
1.7B	4096 / 128 / c8, 3968-token shared prefix	+1.6%	+5.4%
1.7B	8192 / 64 / c4, 8000-token shared prefix	+1.2%	+12.3%
1.7B	16384 / 256 / c2	−3.7%	+12.2%
1.7B	30000 / 128 / c1	−9.4%	+18.3%
8B	64 / 256 / c1	+6.2%	+3.6%
8B	512 / 256 / c32	+1.1%	+3.6%
8B	2048 / 16 / c8	−4.4%	+5.2%
8B	4096 / 4 / c4	−5.8%	+6.3%
8B	4096 / 128 / c8, 3968-token shared prefix	+8.1%	+3.9%
8B	8192 / 64 / c4, 8000-token shared prefix	+5.5%	+6.3%
8B	16384 / 256 / c1 (prefill only)		+13.2%
0.6B	512 / 256 / c64	+1.2%	+6.8%
0.6B	512 / 256 / c128	−2.0%	+18.9%
0.6B	128 / 512 / c128	+1.4%	−2.5%

model	workload (in / out / concurrency)	Δ throughput	Δ TTFT
0.6B	2048 / 128 / c64	−8.0%	+11.3%
1.7B	512 / 256 / c64	+0.7%	+0.8%
1.7B	512 / 256 / c128	−0.3%	+2.3%
1.7B	128 / 512 / c128	+0.9%	−0.1%
1.7B	2048 / 128 / c64	−2.9%	+3.4%
4B	512 / 256 / c64	+6.5%	+2.8%
4B	128 / 512 / c64	+7.5%	+2.8%

Table 5. GSM8K at two client concurrencies, and top-1 agreement with an fp32 HF reference over 1,280 teacher-forced positions (0.6B: 99.53% original, 99.61% PI; 4B: 99.30%, 98.91%). Every disagreement between the modes is at a position whose fp32 top-2 margin is below 0.17 logits.

model	mode	strict acc. c=1	c=16	generations differing, c=1 vs c=16	top-1 agreement with fp32
Qwen3-1.7B (1319)	original	69.37	68.69	888	99.14%
Qwen3-1.7B (1319)	PI	69.07	69.07	**0**	99.30%
Qwen3-8B (500)	original	89.80	89.80	322	99.53%
Qwen3-8B (500)	PI	90.20	90.20	**0**	99.53%

Kernels in detail. Decode, PI's time relative to FlashInfer's: 0.91–1.06× the faster of its CUDA-core and tensor-core decode at batch 8–32; 0.73–0.85× with 8 or 32 requests sharing a 4096-token prefix; 1.05–1.10× at batch 128; 1.03–1.10× at batch 1 with 8K–32K keys and 1.92× (24.8 vs 12.9 µs) at 1024 keys, where two groups give two CTAs per KV head while FlashInfer splits finer. In the engine at batch 1, PI's decode takes 12.9 µs per layer on 8B against 30.7 µs for FlashInfer's decode and merge. Prefill and extend, PI's time: 1.12–1.80× FlashInfer's on 17 of 18 shapes (0.55× on the 18th). Best PI GEMM against cuBLAS over the 20 Qwen3 projections, every launch streaming from DRAM: 0.78–1.37× at $M = 1$ (only 0.6B o and down above 1.05×); 0.43–1.26× at M = 8–64; 0.81–1.33× at M = 128–512; 0.76–1.22× at M = 1024–2048. A decode step's projections replayed as one graph take 1.11×, 0.77× and 1.03× cuBLAS's time at M = 1, 8 and 32 on 0.6B.

Appendix B: Kernel details

Prefill. Q, K and V live in 128-byte-swizzled tiles — a 64-row Q tile plus two 32-key K and V slots, 48 KB, two CTAs per SM. K and V load a full iteration ahead behind one barrier per tile, and positions past the KV length are zero-filled without a global read. The sm_80 register-Q variant moves the fp32 group totals, touched once per 512 keys, to shared memory (80 KB); with 8 warps per CTA it copies each K/V tile once for twice the rows, is 5–8% faster on long and batched prefills and up to 12% slower on short extends, and the launcher takes it only when 128-row CTAs fill two waves. Sixteen-row warps are deliberate: 32-row warps need the group totals outside registers, which halves occupancy, and the built variant ran 1.4–1.8× the shipped kernel's time. Pipelining $QK_{t+1}^{\top}$ behind tile t and 8-warp CTAs brought 30000-token TTFT from +41.8% to +18.3%. For long prefixes with few query rows, a wave-model planner splits a row tile's groups across CTAs and a merge kernel folds them.

Decode. Per tile, each warp computes an 8-key slice of S; the tile maximum is exchanged through shared memory; warp 0 recomputes $\sum P$ in the canonical order; each warp accumulates four of the sixteen output column tiles. In a multi-group request each warp writes its slice of the group partial and counts its arrival on a per-(request, KV head, warp) counter; the last arrival folds that slice over all groups. No CTA-wide barrier follows the arrival: an earlier version that parked the CTA on one atomic doubled kernel time under L2 pressure. The group is the slowest grid index, so working CTAs are dispatched before empty ones; on the A800 this cut Qwen3-1.7B's decode attention at 8 requests from 1001 to 842 µs per step.

GEMM. Balanced kernels exist because decode GEMMs stream weights, and fixed 32-column tiles leave some SMs with twice the bytes of others. CUTLASS kernels on segmented shapes call the mainloop once per segment from a zero accumulator and fold before the stock epilogue; 64-deep stages took several M = 128–512 projections from 1.1–1.3× cuBLAS's time to 0.85–1.05×. Stream-K runs over one worker warp per (SM, warp slot) for $M \leq 8$, and at

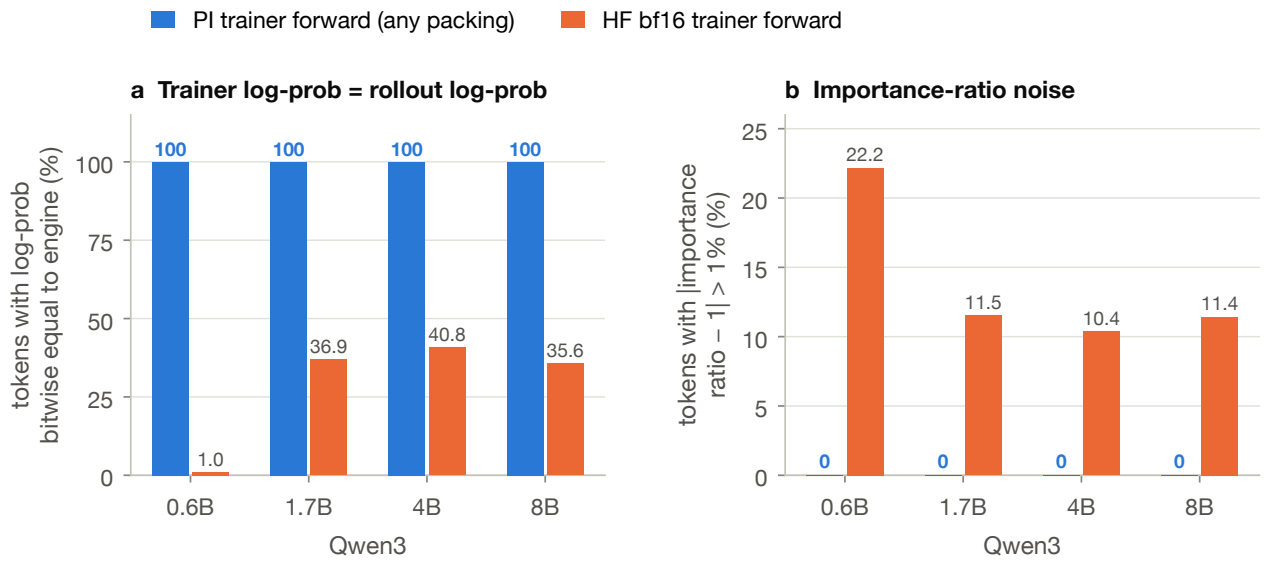


Figure 7. Rollout log-probabilities per model. (a) Tokens whose trainer log-probability equals the engine's bit for bit. (b) Tokens whose importance ratio deviates from 1 by more than 1% (HF: each sequence alone; batches of 8 give 10.6–22.1%).

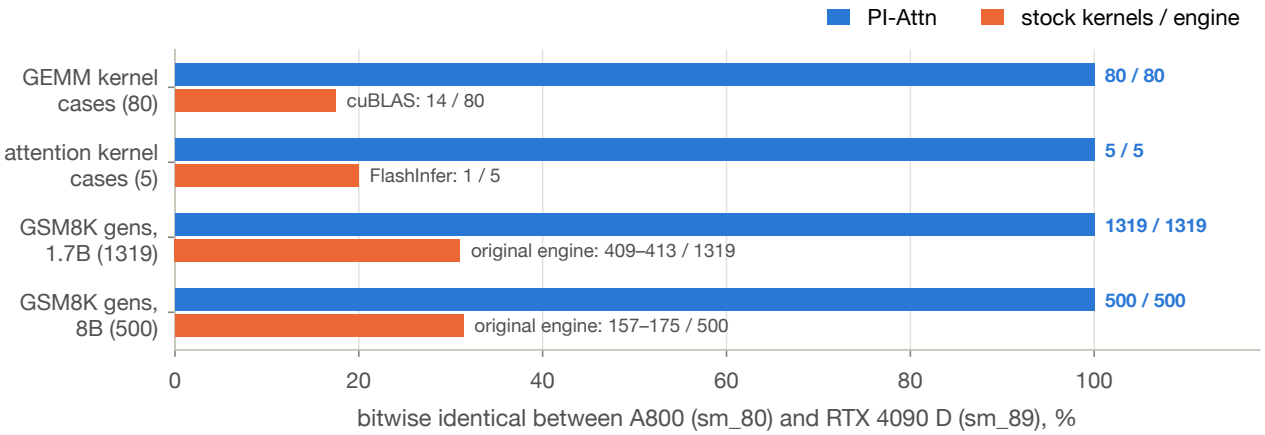


Figure 8. Bitwise identity between an A800 and an RTX 4090 D: kernel cases and GSM8K generations (both client concurrencies; the original engine's range is over the two).

CTA granularity over CUTLASS tiles for mid M, handing the accumulator fragments from CTA to CTA and refusing to launch unless all CTAs can be resident. The segment-parallel kernel gives each (tile, segment) its own CTA, and the last arrival adds the fragments to +0 in ascending order; the segment-split kernel at $M \leq 8$ gives each segment a warp with its own `cp.async` ring and folds through shared memory. The tuner times bursts of graph-captured launches over a ring of weight copies larger than L2, because a microsecond kernel timed after an L2 flush, or launched from Python, is misranked.

Appendix C: RTX 4090 D

The 4090 D ran the full evaluation with an earlier engine. PI passed every detcheck check, at token level, on all four models and all eight server configurations, where the original engine failed 13–14 of 32 comparisons in each. End to end, PI was faster than the original engine on 18 of 20 greedy points and kept time to first token within +1.6% on every long-prompt workload, 16K–30K tokens included.

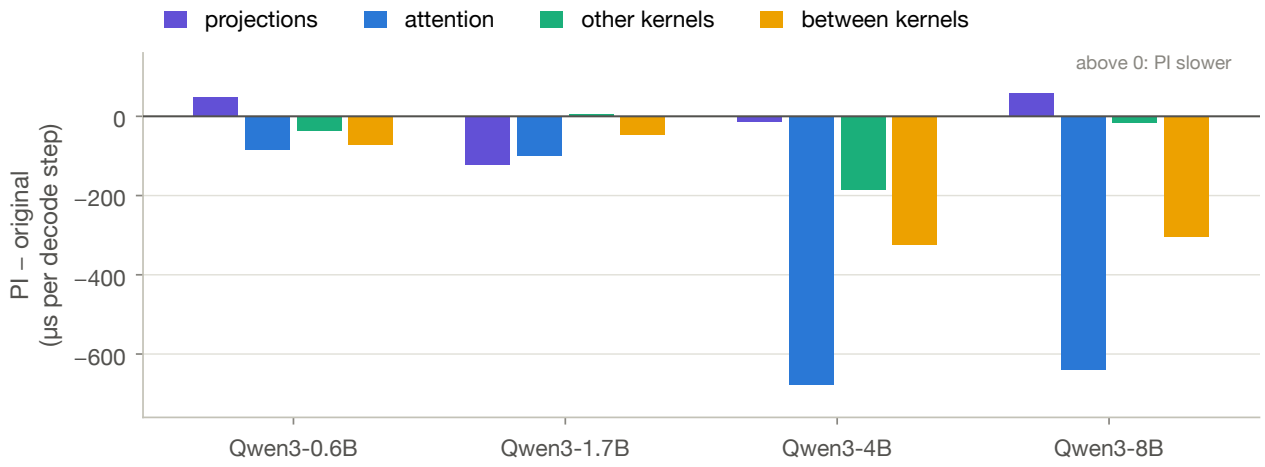


Figure 9. Per-step time of PI minus the original engine at batch 1, by kernel category (node-level trace). Attention is faster on every model, by 86–679 µs per step; projections cost 0.5% of a step on 8B and are faster on 1.7B and 4B. 0.6B's two segmented projections take 9.25 and 10.66 µs against cuBLAS's 7.33 and 9.28: their weights stream in 2–3 µs, so the round trip of the two-segment fold is the gap.

Table 6. End-to-end throughput of PI vs. original on the RTX 4090 D (earlier engine).

model	64/256 c1	512/256 c8	512/256 c32	2048/256 c16	4096/128 c4
0.6B	+2.0%	+8.5%	+4.3%	+5.6%	+5.8%
1.7B	−1.4%	+5.7%	+4.9%	+3.4%	+3.9%
4B	+0.1%	+5.9%	+6.1%	+3.2%	+3.0%
8B	−0.6%	+1.5%	+3.3%	+0.6%	+0.9%

Appendix D: Artifact

Everything is in one repository, `github.com/HaochengLu/pi-attn`: the engine (`llm_infer_rust/`, kernels in `llm_infer_rust/src/kernels/pi/`), the kernel bindings and tests, detcheck, the evaluation scripts, Gated DeltaNet and all results. `eval/run_all.sh` rebuilds the engine and regenerates every result in about 8 hours on one A800, `results/README.md` maps every table and figure to its files, and `paper/figures.py` redraws every figure from them.